

Catalog Comparison Tool

Automated change detection between two dataset versions — Python & Excel

Built a Python-based tool that compares two versions of a product catalog and automatically detects additions, removals, and field-level changes — turning a manual, error-prone review process into a repeatable, few-second workflow.

The tool outputs a structured Excel report with:

- Summary counts of everything that changed
- A per-category / per-model breakdown
- Dedicated tabs for added rows, removed rows, and field-level changes
- Flags for unmatched entries and duplicate keys needing manual review

Tools used: Python (pandas, openpyxl), Excel

Note: this version uses sample data built for demonstration purposes.

Product Catalog Comparison — Demo Outp

Illustrative project built with fictional sample data for portfolio purpo

Metric	Count
Total rows — previous catalog	40
Total rows — current catalog	44
Rows added	7
Rows removed	1
Field-level changes	6
Unmatched vehicle models	1
Ambiguous / duplicate keys	1

Model	Rows (Current)	Added	Removed	Changed
Ridgeline Adventure	2	2	0	0
Solstice LX	9	1	1	4
Solstice Sport	8	1	1	1
Trailhawk Base	9	1	0	0
Trailhawk XT	8	1	1	1
Voyager Premium	8	1	1	1

Part Number	Model	Category	Description	Price
AC-2000	Trailhawk Base	Towing	Towing accessory N	643.66
AC-2001	Voyager Premium	Interior	Interior accessory	459.19
AC-2002	Solstice LX	Interior	Interior accessory N	542.19
AC-2003	Trailhawk XT	Electronics	Electronics accesso	206.98
AC-2004	Solstice Sport	Wheels	Wheels accessory N	575.2
AC-2005	Ridgeline Adventure	Exterior	Exterior accessory	235.38
AC-2006	Ridgeline Adventure	Exterior	Exterior accessory	441.54

Part Number	Model	Field Changed	Previous Value	Current Value
AC-1004	Solstice LX	Description	Wheels accessory 5 for S	Wheels accessory 5 for S
AC-1002	Solstice LX	Price	211.39	241.13
AC-1014	Solstice Sport	Description	Interior accessory 7 for S	Interior accessory 7 for S
AC-1018	Trailhawk XT	Price	534.71	599.44
AC-1005	Solstice LX	Description	Exterior accessory 6 for	Exterior accessory 6 for
AC-1037	Voyager Premium	Description	Exterior accessory 6 for	Exterior accessory 6 for

```
1  """
2  Catalog Comparison Tool
3  -----
4  Compares two versions of a product catalog (e.g. a parts list, price list,
5  or accessory catalog) and generates a structured Excel report highlighting
6  what changed between them.
7
8  Typical use case: a business receives a periodically updated catalog file
9  (new prices, new items, discontinued items) and needs a fast, repeatable
10 way to see exactly what changed since the last version – without manually
11 scanning thousands of rows.
12
13 Workflow:
14     1. Drop the previous version of the catalog into the INPUT_DIR as
15        "previous.xlsx" and the new version as "current.xlsx".
16     2. Run this script.
17     3. Get a single Excel report in OUTPUT_DIR with:
18         - Summary          : high-level counts of what changed
19         - Vehicle_Summary   : change counts broken down by category/model
20         - Added_in_Current  : rows that are new in the latest file
21         - Removed_since_Previous : rows that no longer exist
22         - Changed_Rows      : rows where a field value changed
23         - Unmatched_Entries : rows whose category/model has no prior match
24         - Ambiguous_Keys    : duplicate key values needing manual review
25     4. The previous comparison report (if any) is automatically moved to
26        an ARCHIVE_DIR so nothing gets overwritten.
27
28 Author: Majd Kabbani
29 """
30
31 import os
32 import shutil
33 from datetime import datetime
34
35 import pandas as pd
36 import openpyxl
37 from openpyxl.styles import Font, PatternFill, Alignment
38 from openpyxl.utils import get_column_letter
39
40 # -----
41 # CONFIG – adjust these to match your dataset
42 # -----
```

```
43 INPUT_DIR = "input"
44 OUTPUT_DIR = "output"
45 ARCHIVE_DIR = "archive"
46
47 PREVIOUS_FILE = os.path.join(INPUT_DIR, "previous.xlsx")
48 CURRENT_FILE = os.path.join(INPUT_DIR, "current.xlsx")
49
50 # The column that uniquely identifies a row (e.g. SKU, Part Number, ID)
51 KEY_COLUMN = "Item ID"
52
53 # The column used to group rows for the category-level summary
54 GROUP_COLUMN = "Category"
55
56 # Columns to compare for "changed" detection (besides the key/group columns)
57 COMPARE_COLUMNS = ["Description", "Price"]
58
59 FONT_NAME = "Arial"
60
61
62 # -----
63 # STEP 1 – Load the two catalog versions
64 # -----
65 def load_catalog(path: str) -> pd.DataFrame:
66     df = pd.read_excel(path)
67     df[KEY_COLUMN] = df[KEY_COLUMN].astype(str)
68     return df
69
70
71 # -----
72 # STEP 2 – Compare the two versions
73 # -----
74 def compare_catalogs(previous: pd.DataFrame, current: pd.DataFrame) -> dict:
75     prev_keys = set(previous[KEY_COLUMN])
76     curr_keys = set(current[KEY_COLUMN])
77
78     added_keys = curr_keys - prev_keys
79     removed_keys = prev_keys - curr_keys
80     common_keys = prev_keys & curr_keys
81
82     added_rows = current[current[KEY_COLUMN].isin(added_keys)].copy()
83     removed_rows = previous[previous[KEY_COLUMN].isin(removed_keys)].copy()
84
```

```
85     # Detect field-level changes among rows present in both versions
86     prev_indexed = previous.set_index(KEY_COLUMN)
87     curr_indexed = current.set_index(KEY_COLUMN)
88
89     changed_records = []
90     for key in common_keys:
91         old_row = prev_indexed.loc[key]
92         new_row = curr_indexed.loc[key]
93         for col in COMPARE_COLUMNS:
94             old_val, new_val = old_row.get(col), new_row.get(col)
95             if pd.isna(old_val) and pd.isna(new_val):
96                 continue
97             if old_val != new_val:
98                 changed_records.append({
99                     KEY_COLUMN: key,
100                     GROUP_COLUMN: new_row.get(GROUP_COLUMN, ""),
101                     "Field Changed": col,
102                     "Previous Value": old_val,
103                     "Current Value": new_val,
104                 })
105     changed_rows = pd.DataFrame(changed_records)
106
107     # Ambiguous keys: duplicates within the current file
108     key_counts = current[KEY_COLUMN].value_counts()
109     ambiguous_keys = key_counts[key_counts > 1].reset_index()
110     ambiguous_keys.columns = [KEY_COLUMN, "Occurrences in Current"]
111
112     # Unmatched entries: rows whose group/category never appeared in the
113     # previous file at all (a brand-new model/category, not just a new item)
114     prev_groups = set(previous[GROUP_COLUMN].unique())
115     curr_groups = set(current[GROUP_COLUMN].unique())
116     new_groups = curr_groups - prev_groups
117     unmatched_entries = current[current[GROUP_COLUMN].isin(new_groups)].copy()
118
119     # Per-group summary
120     group_summary = []
121     for group in sorted(curr_groups):
122         rows_current = len(current[current[GROUP_COLUMN] == group])
123         added_ct = len(added_rows[added_rows[GROUP_COLUMN] == group])
124         removed_ct = len(removed_rows[removed_rows[GROUP_COLUMN] == group])
125         changed_ct = len(changed_rows[changed_rows[GROUP_COLUMN] == group]) if not changed_rows.empty else 0
126         group_summary.append({
```



```
127         GROUP_COLUMN: group,
128         "Rows (Current)": rows_current,
129         "Added": added_ct,
130         "Removed": removed_ct,
131         "Changed": changed_ct,
132     })
133     group_summary_df = pd.DataFrame(group_summary)
134
135     return {
136         "added": added_rows,
137         "removed": removed_rows,
138         "changed": changed_rows,
139         "ambiguous": ambiguous_keys,
140         "unmatched": unmatched_entries,
141         "group_summary": group_summary_df,
142         "totals": {
143             "previous_rows": len(previous),
144             "current_rows": len(current),
145             "added": len(added_rows),
146             "removed": len(removed_rows),
147             "changed": len(changed_rows),
148             "ambiguous": len(ambiguous_keys),
149             "unmatched": len(unmatched_entries),
150         },
151     }
152
153
154 # -----
155 # STEP 3 – Write the Excel report
156 # -----
157 def style_header(ws, ncols, row=1):
158     fill = PatternFill("solid", fgColor="1F3864")
159     font = Font(name=FONT_NAME, bold=True, color="FFFFFF")
160     for c in range(1, ncols + 1):
161         cell = ws.cell(row=row, column=c)
162         cell.fill = fill
163         cell.font = font
164         cell.alignment = Alignment(horizontal="center")
165
166
167 def autosize(ws, ncols, width=18):
168     for c in range(1, ncols + 1):
```

```
169         ws.column_dimensions[get_column_letter(c)].width = width
170
171
172 def write_df(ws, df: pd.DataFrame, start_row=1):
173     headers = list(df.columns)
174     for j, h in enumerate(headers, start=1):
175         ws.cell(row=start_row, column=j, value=h)
176     style_header(ws, len(headers), row=start_row)
177     for i, (_, row) in enumerate(df.iterrows(), start=start_row + 1):
178         for j, h in enumerate(headers, start=1):
179             ws.cell(row=i, column=j, value=row[h]).font = Font(name=FONT_NAME, size=10)
180     autosize(ws, len(headers))
181
182
183 def build_report(results: dict, output_path: str):
184     wb = openpyxl.Workbook()
185
186     # --- Summary sheet ---
187     ws = wb.active
188     ws.title = "Summary"
189     ws["A1"] = "Catalog Comparison Report"
190     ws["A1"].font = Font(name=FONT_NAME, bold=True, size=13)
191     ws["A2"] = f"Generated {datetime.now().strftime('%Y-%m-%d %H:%M')}"
192     ws["A2"].font = Font(name=FONT_NAME, italic=True, size=9, color="666666")
193
194     labels = [
195         ("Total rows – previous catalog", results["totals"]["previous_rows"]),
196         ("Total rows – current catalog", results["totals"]["current_rows"]),
197         ("Rows added", results["totals"]["added"]),
198         ("Rows removed", results["totals"]["removed"]),
199         ("Field-level changes", results["totals"]["changed"]),
200         ("New categories/models (unmatched)", results["totals"]["unmatched"]),
201         ("Ambiguous / duplicate keys", results["totals"]["ambiguous"]),
202     ]
203     start = 4
204     ws.cell(row=start, column=1, value="Metric").font = Font(name=FONT_NAME, bold=True, color="FFFFFF")
205     ws.cell(row=start, column=2, value="Count").font = Font(name=FONT_NAME, bold=True, color="FFFFFF")
206     ws.cell(row=start, column=1).fill = PatternFill("solid", fgColor="1F3864")
207     ws.cell(row=start, column=2).fill = PatternFill("solid", fgColor="1F3864")
208     for i, (label, value) in enumerate(labels, start=start + 1):
209         ws.cell(row=i, column=1, value=label).font = Font(name=FONT_NAME, size=10)
210         ws.cell(row=i, column=2, value=value).font = Font(name=FONT_NAME, size=10)
```

```
211     autosize(ws, 2, 34)
212
213     # --- Other sheets ---
214     sheet_map = [
215         ("Vehicle_Summary", results["group_summary"]),
216         ("Added_in_Current", results["added"]),
217         ("Removed_since_Previous", results["removed"]),
218         ("Changed_Rows", results["changed"]),
219         ("Unmatched_Entries", results["unmatched"]),
220         ("Ambiguous_Keys", results["ambiguous"]),
221     ]
222     for name, df in sheet_map:
223         ws = wb.create_sheet(name)
224         if df is not None and not df.empty:
225             write_df(ws, df)
226         else:
227             ws["A1"] = "No rows found for this category."
228             ws["A1"].font = Font(name=FONT_NAME, italic=True)
229
230     wb.save(output_path)
231
232
233     # -----
234     # STEP 4 – Archive the previous report so nothing gets overwritten
235     # -----
236     def archive_previous_report(output_path: str):
237         if os.path.exists(output_path):
238             os.makedirs(ARCHIVE_DIR, exist_ok=True)
239             timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
240             archived_name = f"comparison_report_{timestamp}.xlsx"
241             shutil.move(output_path, os.path.join(ARCHIVE_DIR, archived_name))
242
243
244     # -----
245     # MAIN
246     # -----
247     def main():
248         os.makedirs(OUTPUT_DIR, exist_ok=True)
249         output_path = os.path.join(OUTPUT_DIR, "comparison_report.xlsx")
250
251         archive_previous_report(output_path)
252
```

```
253     previous = load_catalog(PREVIOUS_FILE)
254     current = load_catalog(CURRENT_FILE)
255
256     results = compare_catalogs(previous, current)
257     build_report(results, output_path)
258
259     print(f"Comparison complete. Report saved to: {output_path}")
260     print(f"  Added: {results['totals']['added']}")
261     print(f"  Removed: {results['totals']['removed']}")
262     print(f"  Changed: {results['totals']['changed']}")
263     print(f"  Unmatched: {results['totals']['unmatched']}")
264     print(f"  Ambiguous keys: {results['totals']['ambiguous']}")
265
266
267 if __name__ == "__main__":
268     main()
269
```